PROJECT TITLE	**:**	DOC-GENIE
TEAM ID	**:**	69a92a5e2bcee27bd8d0f243
TEAM MEMBERS	**:**	PARVATHI P G KENSIYA M SASI SUREKSHA S SUBASREE N
COLLEGE NAME	**:**	S. A. V SAHAYA THAI ARTS AND SCIENCE (WOMEN) COLLEGE
COLLEGE CODE	**:**	138
DATE	**:**	07.03.2026

DOC-GENIE

AI-Powered Python Documentation Generator with

IBM Granite 3.32B

PROJECT DESCRIPTION:

Doc-Genie is an intelligent documentation generation system designed to automatically create professional docstrings for Python functions. The system uses Artificial Intelligence and code analysis techniques to help developers generate clear and structured documentation without manually writing it.

The project is developed using Python and integrates the IBM Granite 3.32B Instruct model along with the Gradio framework to provide an interactive web-based interface. The system analyzes Python functions entered by the user or uploaded through a Python (.py) file and automatically generates detailed docstrings based on the function's structure, parameters, and logic.

The main objective of this project is to simplify the process of writing documentation for programmers, students, and software developers. Writing proper documentation is an important part of software development, but it can be time-consuming and sometimes difficult. Doc-Genie helps solve this problem by automatically generating accurate documentation using code analysis and AI-based techniques.

The system works by analyzing the Abstract Syntax Tree (AST) of the Python function to extract important details such as function name, parameters, default values, return types, loops, conditions, and mathematical operations. Based on this analysis, the system generates meaningful and professional docstrings that explain the functionality of the code.

Doc-Genie supports multiple documentation styles, including Google style and NumPy style docstrings, allowing developers to choose the format that best suits their coding standards. The generated documentation can also be exported as PDF or TXT files for easy sharing and record keeping. Additionally, the system provides social sharing features that allow users to share generated documentation links through platforms like WhatsApp and Facebook.

By combining AI technology with static code analysis and a user-friendly interface, Doc-Genie provides a powerful and efficient solution for generating high-quality Python documentation.

SCENARIOS

Scenario 1: Automatic Documentation for Python Functions

A Python developer writes multiple functions in a project but forgets to include proper documentation. Writing docstrings manually for each function can be time-consuming and repetitive.

Using the Doc-Genie system, the developer simply copies and pastes the Python function into the application interface or uploads the Python (.py) file. The system analyzes the function structure, including parameters, return values, loops, and conditions. Based on this analysis, Doc-Genie automatically generates a professional docstring explaining the purpose of the function, the parameters used, and the return value.

The developer can choose between Google-style or NumPy-style documentation formats. This allows the developer to quickly produce clear and structured documentation without manually writing it.

Scenario 2: Documentation Support for Students Learning Python

A computer science student is learning Python programming and writing several functions as part of assignments or projects. However, the student may not know how to properly write documentation for their code.

By using the Doc-Genie interface, the student enters the Python function code into the system. The application analyzes the logic of the function and automatically generates a clear docstring explaining how the function works, including its parameters and output.

The generated documentation helps the student understand proper documentation standards used in real software development. It also improves code readability and helps teachers easily understand the student's program.

Scenario 3: Generating Project Documentation

Software developers need documentation for Python functions, but writing it manually can take time.

Using the Doc-Genie system, the developer enters the function code or uploads a Python (.py) file, and the system automatically generates docstrings explaining the function.

ARCHITECTURE OVERVIEW

The Doc-Genie system is designed using a modular architecture that combines Python programming, code analysis techniques, AI model integration, and web interface development.

Python is used as the core programming language for implementing the system logic. The application analyzes Python functions using the Abstract Syntax Tree (AST) module to understand the function structure, parameters, return values, loops, and conditions, which helps in generating accurate documentation.

The user interface is developed using the Gradio framework, which provides an interactive web-based environment where users can enter Python function code or upload Python (.py) files. The system then processes the code and generates professional docstrings in different formats.

The application also includes modules for documentation generation, file processing, and export functionality. Users can generate docstrings in Google-style or NumPy-style formats and export the generated documentation as PDF or TXT files.

The modular design of the system makes it easy to maintain, update, and extend with additional features in the future.

Core Technologies

- Python 3.x: Core programming language used to implement the application logic and code analysis.
- IBM Granite 3.3 2B Instruct: AI model used for intelligent content generation and documentation assistance.
- Gradio: Framework used to create the interactive web interface for user interaction.
- AST (Abstract Syntax Tree): Used for analyzing Python function structure and extracting parameters and logic.
- ReportLab: Library used for generating downloadable PDF documentation reports.
- Google Colab: Cloud-based environment used for development and testing of the project.

PRE-REQUISITES

Software Requirements

- Python 3.8+: Required to run the core application and implement the documentation generation logic.
- Google Account: Required to access **Google Colab** for running and testing the project.
- Gradio: Used to build the interactive web interface *(install using: pip install gradio)*.
- Transformers Library: Required to load and use the IBM Granite AI model *(pip install transformers)*.
- Web Browser: Chrome, Firefox, Safari, or Edge for accessing the application interface.
- Internet Connection: Required for Google Colab and AI model access.

Knowledge Prerequisites

To understand the project implementation, basic knowledge of the following topics is useful:

- Python programming fundamentals
- Basic understanding of Python functions
- Knowledge of Docstrings and code documentation
- Basic understanding of AI models
- Familiarity with Google Colab

These concepts help users understand how the system analyzes Python functions and generates documentation automatically.

Hardware Requirements

The hardware requirements for this project are minimal because most of the processing is performed in the cloud environment.

Minimum system requirements include:

- Processor: Intel i3 or equivalent
- RAM: Minimum 4 GB
- Storage: At least 1 GB free space
- Display resolution: 1280 × 720 or higher
- Internet connection

PROJECT WORKFLOW

The development of the Doc-Genie system follows a structured workflow that includes environment setup, core development, interface implementation, documentation generation, and testing. Each phase focuses on building specific components of the application.

The workflow ensures that the system is developed in a modular and organized manner. This makes the application easier to maintain, test, and improve in the future.

The main phases of the project include:

- Environment Setup and Model Configuration
- Core Application Development
- Gradio Interface Development
- Deployment
- Testing and Validation

Each phase contributes to building a complete AI-assisted Python documentation generation system.

Phase 1: Environment Setup & Model Configuration

This phase focuses on preparing the development environment and installing the necessary libraries required for the project.

The application is developed using **Google Colab**, which provides a cloud-based environment for executing Python code and running machine learning libraries. This platform simplifies development by providing pre-configured tools and computing resources.

During this phase, the required Python libraries such as **Gradio, Transformers, Torch, and ReportLab** are installed. These libraries are used for building the web interface, processing code, generating documentation, and exporting reports.

Activity 1.1: Install Dependencies and Setup Environment

Before implementing the application, the required Python packages must be installed. These packages provide the necessary functionality for code analysis, documentation generation, and user interface development.

The required libraries include:

- Gradio
- Transformers
- ReportLab
- Torch (PyTorch)

The following command is used to install the dependencies:

```
!pip install -U transformers
Requirement already satisfied: transformers in /usr/local/lib/python3.12/dist-packages (5.0.0)
Collecting transformers
  Downloading transformers-5.3.0-py3-none-any.whl.metadata (32 kB)
Requirement already satisfied: huggingface-hub<2.0,>=1.3.0 in /usr/local/lib/python3.12/dist-packages (from transformers) (1.5.0)
Requirement already satisfied: numpy>=1.17 in /usr/local/lib/python3.12/dist-packages (from transformers) (2.0.2)
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.12/dist-packages (from transformers) (26.0)
Requirement already satisfied: pyyaml>=5.1 in /usr/local/lib/python3.12/dist-packages (from transformers) (6.0.3)
Requirement already satisfied: regex!=2019.12.17 in /usr/local/lib/python3.12/dist-packages (from transformers) (2025.11.3)
Requirement already satisfied: tokenizers<=0.23.0,>=0.22.0 in /usr/local/lib/python3.12/dist-packages (from transformers) (0.22.2)
Requirement already satisfied: typer in /usr/local/lib/python3.12/dist-packages (from transformers) (0.24.1)
Requirement already satisfied: safetensors>=0.4.3 in /usr/local/lib/python3.12/dist-packages (from transformers) (0.7.0)
Requirement already satisfied: tqdm>=4.27 in /usr/local/lib/python3.12/dist-packages (from transformers) (4.67.3)
Requirement already satisfied: filelock>=3.10.0 in /usr/local/lib/python3.12/dist-packages (from huggingface-hub<2.0,>=1.3.0->transformers) (3.24.3)
Requirement already satisfied: fsspec>=2023.5.0 in /usr/local/lib/python3.12/dist-packages (from huggingface-hub<2.0,>=1.3.0->transformers) (2025.3.0)
Requirement already satisfied: hf-xet<2.0.0,>=1.2.0 in /usr/local/lib/python3.12/dist-packages (from huggingface-hub<2.0,>=1.3.0->transformers) (1.3.1)
Requirement already satisfied: httpx<1,>=0.23.0 in /usr/local/lib/python3.12/dist-packages (from huggingface-hub<2.0,>=1.3.0->transformers) (0.28.1)
Requirement already satisfied: typing-extensions>=4.1.0 in /usr/local/lib/python3.12/dist-packages (from huggingface-hub<2.0,>=1.3.0->transformers) (4.15.0)
Requirement already satisfied: click>=8.2.1 in /usr/local/lib/python3.12/dist-packages (from typer->transformers) (8.3.1)
Requirement already satisfied: shellingham>=1.3.0 in /usr/local/lib/python3.12/dist-packages (from typer->transformers) (1.5.4)
Requirement already satisfied: rich>=12.3.0 in /usr/local/lib/python3.12/dist-packages (from typer->transformers) (13.9.4)
Requirement already satisfied: annotated-doc>=0.0.2 in /usr/local/lib/python3.12/dist-packages (from typer->transformers) (0.0.4)
Requirement already satisfied: anyio in /usr/local/lib/python3.12/dist-packages (from httpx<1,>=0.23.0->huggingface-hub<2.0,>=1.3.0->transformers) (4.12.1)
Requirement already satisfied: certifi in /usr/local/lib/python3.12/dist-packages (from httpx<1,>=0.23.0->huggingface-hub<2.0,>=1.3.0->transformers) (2026.2.25)
```

After installing these libraries, the development environment becomes ready for implementing the project.

Activity 1.2: Configure AI Model

In this step, the **IBM Granite 3.3 2B Instruct model** is loaded using the Transformers library. The model is capable of generating structured text responses based on prompts provided by the user.

The tokenizer converts input text into tokens that the model can process. The AI model then generates documentation content based on the given prompt and code input.

The model is configured to automatically use available hardware resources such as **GPU or CPU**. This configuration ensures efficient performance when generating documentation for Python functions.

Phase 2: Core Application Development

This phase focuses on building the main components of the system including Python function analysis, docstring generation, and code processing modules.

These components work together to analyze Python functions, extract important details such as parameters and return values, and generate structured documentation automatically.

Activity 2.1: Python Function Analysis

The system includes a Python function analysis module that examines the structure of the code provided by the user.

This module analyzes important elements of the function such as the function name, parameters, return values, loops, and conditional statements. The analysis is performed using the **Abstract Syntax Tree (AST)** to understand the logic of the function.

- **Google Style**
- **NumPy Style**

When a user selects a documentation style, the system automatically generates the docstring in the selected format. This feature allows developers to create documentation that follows standard coding practices and improves code readability.

This feature allows users to customize the appearance of their website easily.

```python
# ============================================================================

class DocGenieAnalyzer:
    """Analyze Python functions and generate professional docstrings"""

    @staticmethod
    def extract_function_signature(code):
        """Extract function name, parameters, return type from code"""
        try:
            tree = ast.parse(code)
            func_def = tree.body[0]
            if not isinstance(func_def, ast.FunctionDef):
                return None, None

            func_name = func_def.name
            params = []

            for arg in func_def.args.args:
                param_type = "Any"
                if arg.annotation:
                    param_type = ast.unparse(arg.annotation) if hasattr(ast, 'unparse') else "Any"
                params.append({
                    'name': arg.arg,
                    'type': param_type,
                    'default': None
                })
```

Activity 2.2: Docstring Generation Module

The Docstring Generation module is responsible for creating structured documentation for Python functions. This module analyzes the function details and generates appropriate docstrings automatically.

The generated documentation includes important sections such as:

- Function Description
- Parameters
- Return Values

The function description explains the purpose of the function. The parameters section describes the input arguments used in the function, while the return section explains the output produced by the function.

This structure ensures that the generated documentation follows standard coding practices and improves code readability.

```python
def generate_docstring(code, style="google"):
    """Generate docstring for Python function with accurate analysis"""

    if not code.strip():
        return None, "❌ Empty code provided!"

    try:
        signature, func_def = analyzer.extract_function_signature(code)
        if not signature:
            return None, "❌ Invalid Python function code!"

        # Deep analysis of the function
        analysis = analyzer.analyze_function_logic(func_def, code)

        if style.lower() == "google":
            docstring = analyzer.generate_google_docstring(signature, analysis, code)
        else:
            docstring = analyzer.generate_numpy_docstring(signature, analysis, code)

        # Extract function definition line
        code_lines = code.split('\n')
        func_line = code_lines[0]
```

Activity 2.3: Documentation Export Module

To make the generated documentation easy to store and share, the system includes a documentation export module. This module allows users to download the generated docstrings in different file formats.

The export feature supports the following formats:

- **PDF File**
- **TXT File**

The PDF export provides a well-structured report containing the original code and the generated docstring. The TXT export stores the documentation in a simple text format for easy reference.

By providing automatic export options, the system ensures that users can save and share the generated documentation easily without additional effort.

```
pdf_btn = gr.Button("📄 Download PDF", size="sm", variant="primary")
txt_btn = gr.Button("📄 Download TXT", size="sm", variant="primary")

pdf_file = gr.File(label="PDF File", visible=True)
txt_file = gr.File(label="TXT File", visible=True)

export_status = gr.Textbox(label="Export Status", interactive=False, lines=2)

pdf_btn.click(download_pdf, outputs=[export_status, pdf_file])
txt_btn.click(download_txt, outputs=[export_status, txt_file])

gr.Markdown("---")
gr.Markdown("### 📤 Share")

whatsapp_btn = gr.Button("💬 WhatsApp", size="sm", variant="secondary")
facebook_btn = gr.Button("👍 Facebook", size="sm", variant="secondary")
```

Phase 3: Gradio Interface Development

The **Doc-Genie** application uses **Gradio** to build an interactive user interface.

Gradio allows developers to create simple web-based interfaces for AI and Python applications. Through this interface, users can interact with the system and generate documentation for Python functions without writing additional code.

The interface is designed with different sections to organize the main functionalities of the application.

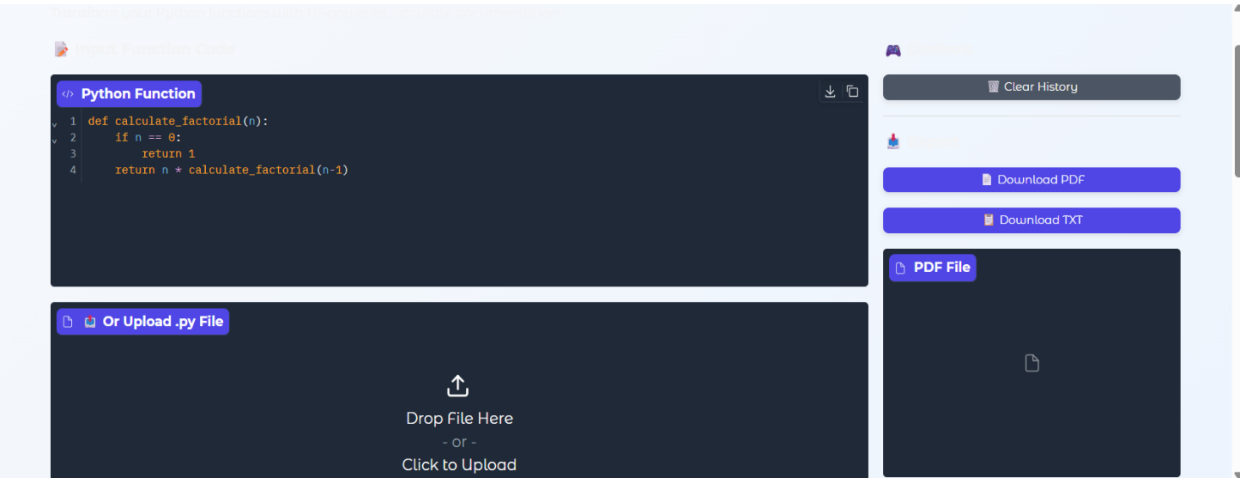
Tab 1: Generate Docstring

This section allows users to generate documentation for Python functions.

Users can enter:

- **Python function code**
- **Upload Python (.py) file**
- **Select docstring style (Google or NumPy)**

Once the user clicks the **Generate Docstring** button, the system analyzes the function and automatically generates a structured docstring explaining the function details.


Python Function
def calculate_factorial(n):
if n == 0:
return 1
return n * calculate_factorial(n-1)
Or Upload .py File
Drop File Here
- or -
Click to Upload
Clear History
Download PDF
Download TXT
PDF File

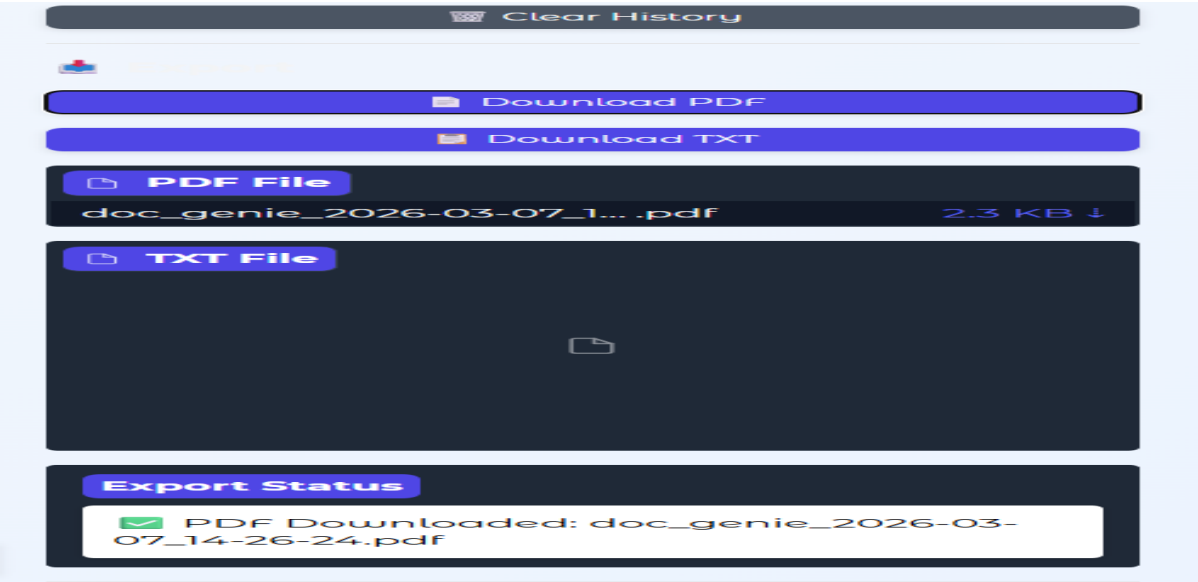
Function with Docstring
Status
Export Status
WhatsApp
Facebook
Share Link

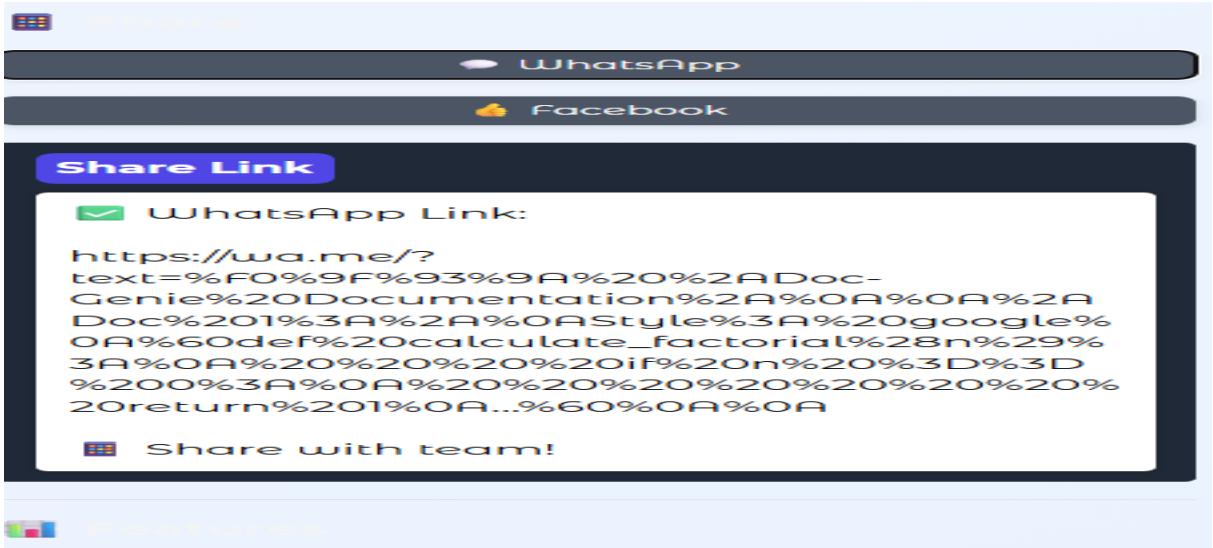
Tab 2: Export and Sharing

This tab allows users to export and share the generated documentation.

Users can download the generated docstrings in different formats such as **PDF** or **TXT** files for future reference. The system also provides options to generate **sharing links** that allow users to share the documentation through platforms like WhatsApp or Facebook.

This feature helps users easily save and share the generated documentation with team members or for project reports.





Tab 3: Features Guide

This tab provides information about the structure of the generated documentation. It explains the main parts included in the generated docstring such as:

Function Description – explains the purpose of the function
Parameters Section – describes the input arguments used in the function
Return Section – explains the output returned by the function

This guide helps users understand how the generated documentation is structured.

Phase 4: Deployment

After implementing all system components, the application is deployed using **Google Colab** and **Gradio sharing functionality**.

Gradio generates a public link that allows users to access the application from any web browser. This link can be shared with others so they can use the **Doc-Genie documentation generator**.

Deployment through Colab makes the application accessible without installing additional software on the user's computer.

Phase 5: Testing and Validation

Testing is performed to ensure that the system works correctly and generates accurate documentation.

Functional testing verifies that:

- **User inputs are accepted correctly**
- **Python functions are analyzed successfully**
- **Docstrings are generated in Google and NumPy styles**
- **Export options generate valid PDF and TXT files**
- **The interface displays the generated documentation correctly**

Different function inputs are tested to confirm that the system behaves as expected.

Conclusion

The **Doc-Genie** project demonstrates how intelligent systems can simplify the process of writing documentation for Python programs. By integrating Python code analysis with the IBM Granite 3.3 2B Instruct model and the Gradio interface, the system allows users to automatically generate structured docstrings for Python functions.

The application analyzes the function structure, including parameters, return values, loops, and conditions, to generate meaningful documentation. It also supports different documentation styles such as **Google style** and **NumPy style**, allowing developers to follow standard coding practices.

Overall, the project shows how AI-assisted tools can help programmers create clear and structured documentation quickly, improving code readability, maintainability, and development efficiency.

SMARTBRIDGE
Let's Bridge the Gap

SKILL WALLET
A SmartBridge Product